Matteo Hu 23094 Laraib Ahmed 22509 Stefano Pugnani 22582

Value and Wage Prediction: Football Players

Machine Learning Project

Academic Year: 2024/2025

Group Id: 6

Table of Contents

1. Problem and dataset description 3
General Description of the Features 3
2. Data Preprocessing 4
One-Hot-Encoding 4
Frequency Encoding 5
Standardization 5
Exploratory Data Analysis 5
3. Algorithms implemented 8
3.1 Multi-Output Regressors 8
3.2 Multi-Task Regressors 8
4. Hyperparameter Optimization 9
Default vs Best Hyperparameters differences: 10
5. Evaluation Results (R^2 Score Comparison) 10
6. Comparison of Models 11
Multi-Output vs Multi-Task Setting: 11
Final considerations: 12

1. Problem and dataset description

For our Machine Learning project, we decided to use the "**Football Player's Market Value**" dataset, retrieved from GitHub at the following address:
https://github.com/Bouaskaoun/Football-Player-s-Market-Value/blob/main/player_list.csv

The dataset contains detailed information about football players, such as physical attributes, economic attributes and skill player attributes. Our goal is to predict two main attributes: the market value and the wage. This regression task is based on numerical and categorical features.

Information about the dataset:

- **Number of samples**: 17,780
- **Number of different features**: 64
- **Target variables**:
  - Value: the estimated value of a player on the market
  - Wage: the salary of a player

General Description of the Features

Key features:
Name (String), Age(Integer), Overall(Integer), Potential(Integer), Team(String), Height(String), Weight(String), Foot(String), Value(String), Wage(String), Release_Clause(String).

Skills and Stats Features:

Additionally, the dataset contains various technical and physical attributes, such as:

- Attacking: Crossing, Finishing, Heading_Accuracy, Short_Passing, Volleys
- Skill: Dribbling, Curve, FK_Accuracy, Ball_Control, Long_Passing
- Movement: Acceleration, Sprint_Speed, Agility, Reactions, Balance
- Power: Shot_Power, Jumping, Stamina, Strength, Long_Shot
- Mentality: Aggression, Interceptions, Positioning, Vision, Penalties, Composure
- Defending: Marking, Standing_Tackle, Sliding_Tackle
- Goalkeeping: GK_Diving, GK_Handling, GK_Kicking, GK_Positioning, GK_Reflexes

There are also aggregate scores like Total_Stats, Base_Stats, and FIFA-specific synthetic features like PAC, SHO, PAS, DRI, DEF, and PHY.

We aim to use this dataset to build some regression models, and compare them, capable of predicting a football player's market value and wage, learning from the various personal, performance-related, and club-related attributes.

In particularly we will train and test the following regressors:

- decision tree regression
- linear regression
- Ridge regression
- Support vector machine

All those cited above use the multi-output setting to predict both the value and wage.

- Multi-task lasso
- Multi-task elastic-net

1. Data Preprocessing

In this section, we aim to clean the dataset to make it easier to handle, by cleaning and selecting the features of our interest.

Originally, the data set contained 64 features. However, not all of them are of our interest. To accomplish our goal, we have selected some specific features:

-Numerical Attributes

- Age: The age of a player is correlated with its value. The higher it is, the lower the value.
- Overall: The overall of a player influences their value.
- Potential: The potential of a player influences their value.
- Player's Stats: The higher the stats of a player are, the higher is their value.

-Categorical Attributes:

- Team: Used to check interesting correlations.
- Height: Used to check interesting correlations.
- Weight: Used to check interesting correlations.
- Foot: Used to check interesting correlations.
- Value: Used to make predictions of our interests.
- Wage: Used to make predictions of our interests.
- Release_Clause: Can influence the Value and Wage of a Player.

One-Hot-Encoding

We have applied the One-Hot-Encoding for the attributes "Foot" and "Position". To make it computable for the model, we have decided to use this type of encoding because:

- In the case of the attribute “Foot”, since it can only acquire two values (Left or Right), using this type of encoding is optimal by keeping the number of features low. In the case of “Position”, more values are possible, however the number of features is still manageable.
- Works well with the regression classifier that we are going to use.
- It avoids wrong interpretation. For instance, with Label Encoding, if GK=0, CB=1, ..., it is possible to interpret wrongly CB>GK.

Frequency Encoding

The reasons why we have used the Frequency Encoding for the feature “Team” are:

- Since there are a vast number of possible teams, One-Hot-Encoding would create a large number of features, such that it would make the work more costly. Frequency Encoding allows us to work only with probability values.
- Label Encoding creates a false order, as previously cited.
- Frequency Encoding allows us to keep the meaningful data compactly

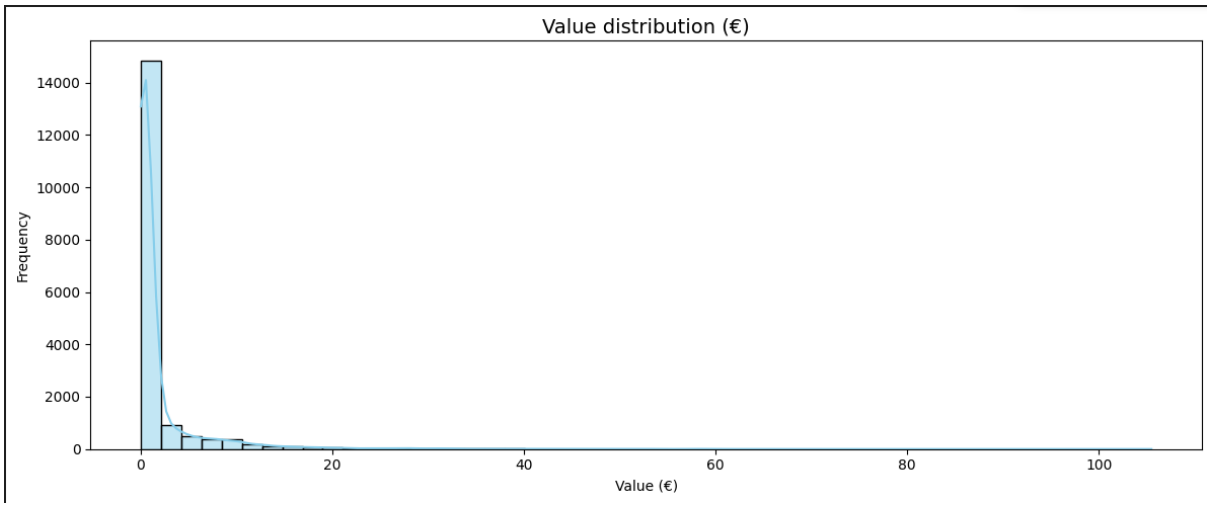
Standardization

Standardization is used in our case, to avoid the values of some features dominating others. For instance, the wage of a player may dominate their height. In our case, we have applied the standardization of the attributes: “Height”, “Weight”, “Value”, “Wage”, “Release_Clause”, “PAC”, “SHO”, “PAC”, “DRI”, “DEF”, “PHY”.

Exploratory Data Analysis

In this phase of the project, our goal is to give a visualization of the current data. The visualization techniques used in the project are Histograms and Box Plots.

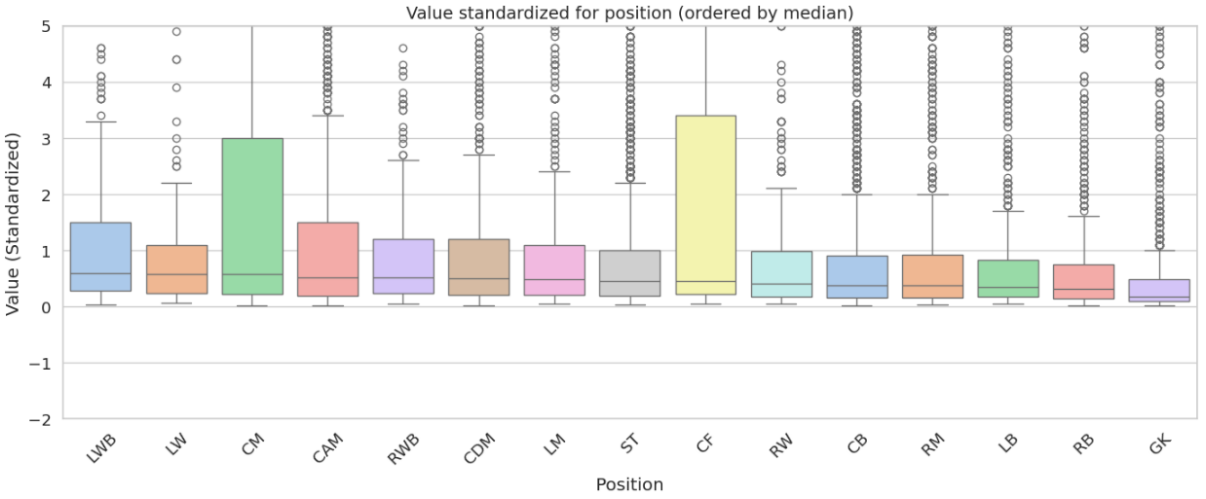
Histogram for Value Distribution with the frequency



Histogram for Wage Distribution with the frequency



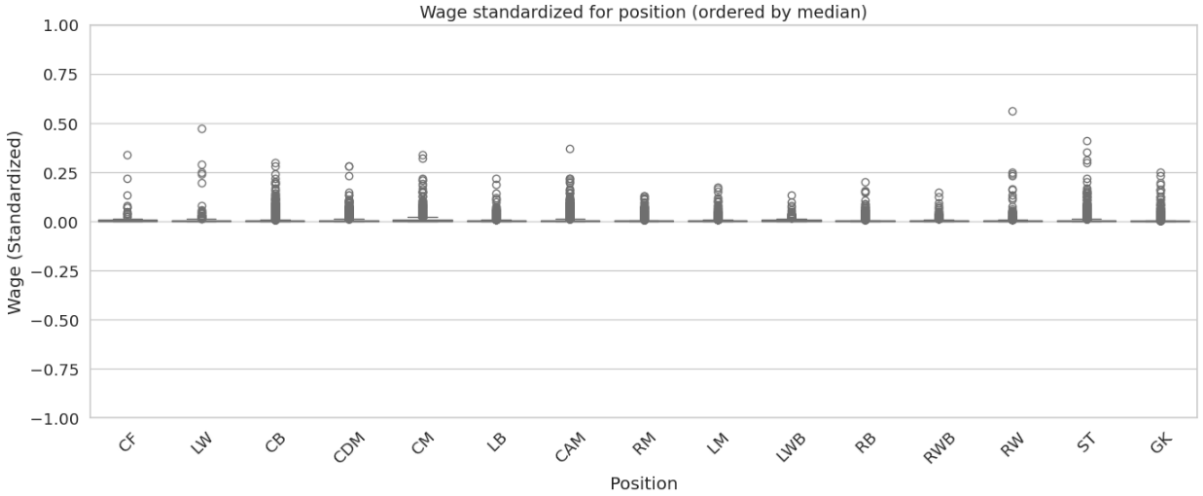
Box Plot for Values based on Position



Observation

- The values for ”CF” and ”CAM” are more well distributed with no outlier.
- CF (yellow column) has the highest standardized median value. That implies that players of this position are the most expensive.
- Defensive roles have lower values rather than others
- External roles, such as LW and RB, have a lower median value rather than CF. However, they have significant outlier. That means that this role has the most expensive players.

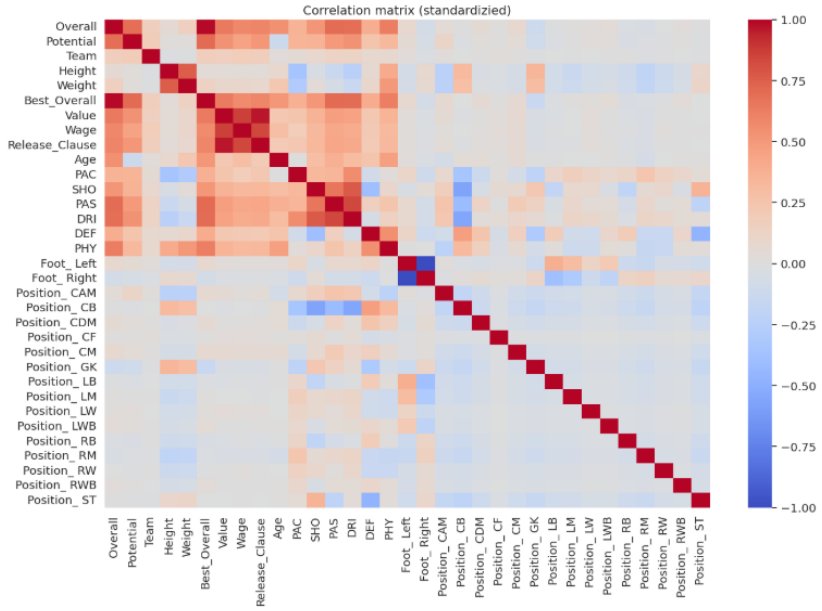
Box Plot for Wages based on Position



Observation

- The differences between positions are less pronounced. Higher median wages are observed for “ST” and “GK”, but overall, the wage distribution appears more uniform.

Heat Map



Observation:

- Value, Wage and Release_Clause are strongly correlated with each other. They are also correlated with other attributes, such as Overall and Potential.
- In the left part above, it can be noticed a significantly more heated part. This means that the selected attributes are correlated to Value and Wage. Moreover, Age is correlated to the Potential and the Overall.

3. Algorithms implemented

In this part we will describe the regression algorithms implemented. We used traditional regression models and advanced multi-output and multi-task approaches. We evaluated their performance using the R^2 score and conducted hyperparameter tuning for each model thanks to the GridsearchCV.

3.1 Multi-Output Regressors

We used the multi output regressor to combine the regressor that normally can predict only one value to predict both of them (the Value and the Wage). In particular we used the MultiOutputRegressor wrapper to adapt standard regressors for our purpose.

We trained the following models:

- **Linear Regression**: A simple linear model that assumes a linear relationship between input features and target variables. This is the only model who doesn't have a hyperparameter.
- **Ridge Regression**: A linear model with L2 regularization to prevent overfitting. GridSearchCV was used to tune the alpha parameter.
- **Decision Tree Regressor**: A non-linear model that splits data into branches to minimize prediction error. Parameters tuned included max_depth and min_samples_split.
- **Support Vector Regressor (RBF Kernel)**: A non-linear model effective in high-dimensional spaces. GridSearchCV was used to optimize C and epsilon.

Training Strategy: All models were trained using a train-test split (80%-20%) with standardized features. GridSearchCV (3-fold) we applied it to each hyperparameter mentioned before.

3.2 Multi-Task Regressors

We also used multi task models, so that we could perform join regression on multiple attributes by learning shared representations.

We implemented the following models:

- **MultiTaskLasso**: A model with L1 regularization that enforces sparsity and selects only the most informative features. GridSearchCV was applied to tune alpha.
- **MultiTaskElasticNet**: A hybrid of L1 and L2 regularization. GridSearchCV was applied to tune both alpha and l1_ratio.

The advantage of this models is to benefit from the correlation between value and wage as seen from the heatmap in the EDA.

Training Strategy: We used again the same method as before so a train-test split (80%-20%) with standardized features. GridSearchCV (3-fold) was applied it to each hyperparameter mentioned before.

4. Hyperparameter Optimization

Hyperparameter optimization was necessary to obtain a optimal model while at the same time preventing overfitting or underfitting. We used, as mentioned above, GridSearchCV from Scikit-learn with 3 fold cross validation to explore the combination of hyperparameter for each model used. This process was used on each regressor except for the linear regression model. The optimization aimed to find the best hyperparameter for predicting both the value and the wage. We used the R^2 score as the evaluation metric.

The R^2 score (alsa named coefficient of determination) is one of the most used metrics to evaluate the quality of a regression model, it is computed in the following manner:

$R^2 = 1 - (RSS / TSS)$

Where RSS is the Residual Sum of Sqaures, so the sum off all the errors between the value predicted and the real value. TSS is the Total Sum of Squares, so it evaluates the variance between the datas and their respective media.

When:

$R^2 = 1$ then the model predict correctly all the value

$R^2 = 0$ then the model is equal the media of the target (so is useless)

$R^2 < 0$ then the model is worse then the media.

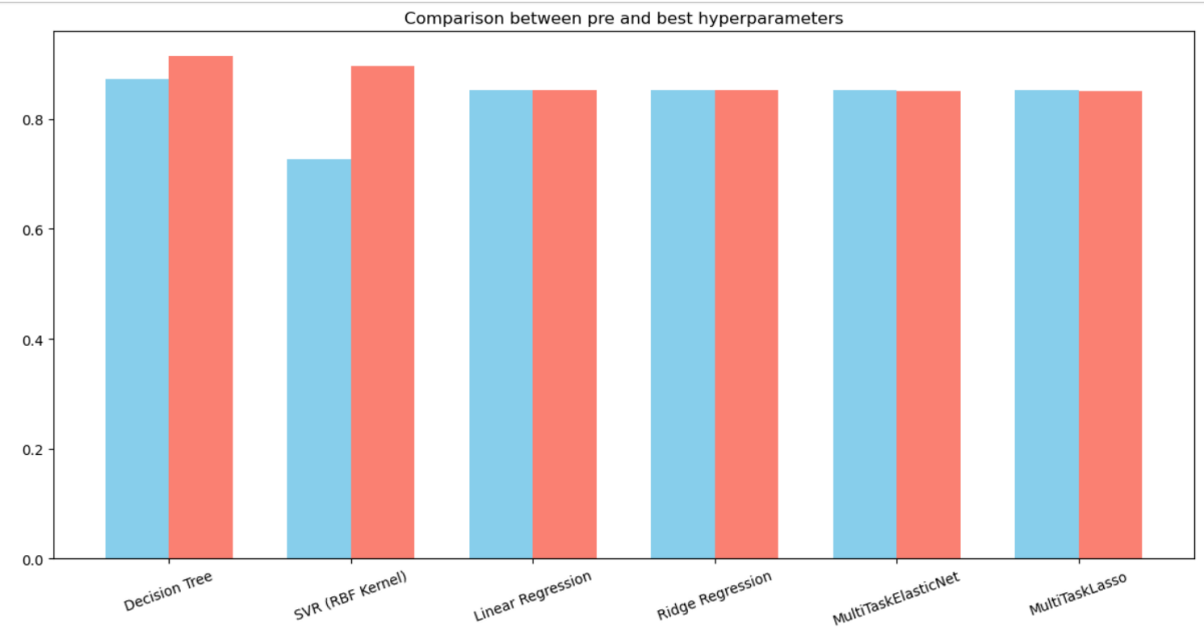
The R2 metrics is the preferable one for our project, because we have two variable to predict that are continued, and because our goal is to predict in the best way the two variable the R2, that is a independent unit, let us to compare models also when the variables are standardizzated and it also says to us how much part of the variance of the target is explained by the model. For example if we have a player which value on the market is 2 milions and the average of the value is 1.50 milions, then if our model predicts 1.5 milions R2 will be 0, if it predicts 2.1 milions then it will be around 1 and when is less than 1.5 then it will be a negative number.

Below is a summary of the hyperparameters tuned and the best values found:

Model	**Tuned Parameters**	**Best parameter**
Decision Tree Regressor	Max depth min sample split	max_depth = 5, min_samples_split = 4
Ridge Regression	Alpha	alpha = 5.0

SVR (RBF Kernel)	C epsilon	C = 100 epsilon: 0.1
Linear Regression	None	None
MultiTaskLasso	Alpha	Alpha = 0.01
MultiTaskElasticNet	Alpha L1_ratio	Alpha = 0.01 0.1

Default vs Best Hyperparameters differences:



The skyblue bar describes the R2 values using pre hyperparameter (default hyperparameter) and the red bar describes the tuned hyperparameter. We can clearly see differences in the first two bars show clear improvement after tuning the hyperparameters, while the rest of the models are already near perfect, without tuning. The SVR (RBF Kernel) model showed the most improvement using the C and epsilon tuning parameters.

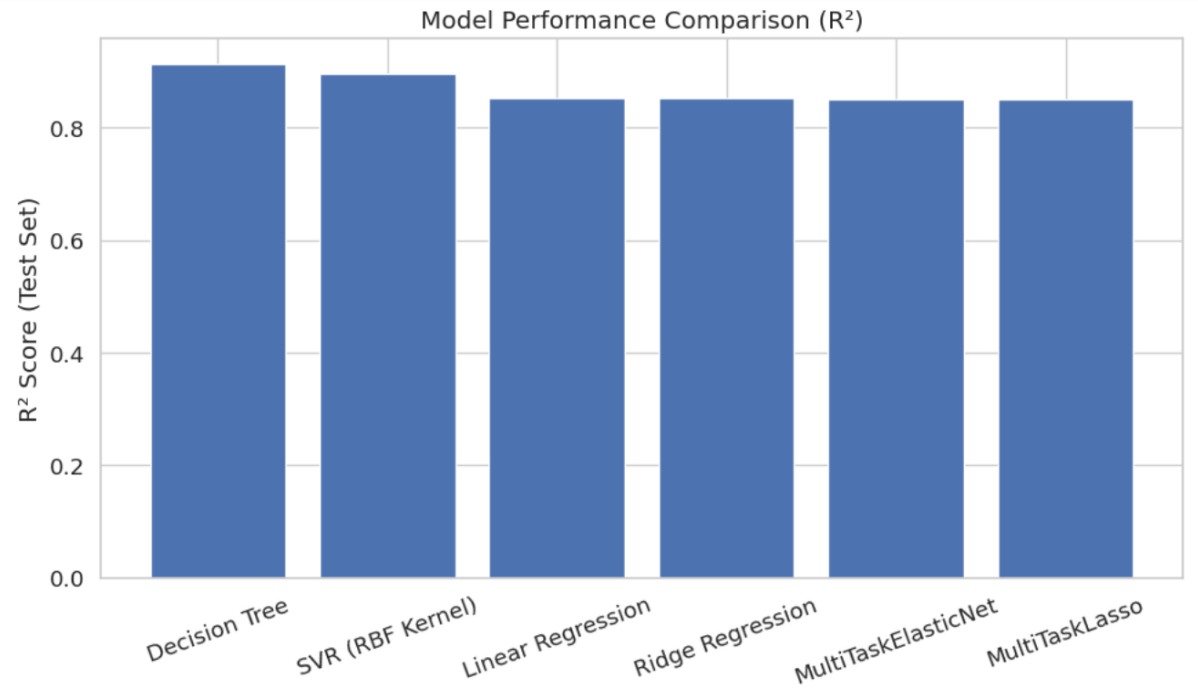
5. Evaluation Results (R^2 Score Comparison)

After training and tuning all the models, we evaluated their performance using the R^2 score on the test set. The table below summarizes the final R^2 values obtained for each model:

R2 scores with each model:

Model	**R2 score**
Decision Tree Regressor	0.9139
SVR (RBF Kernel)	0.8967
Linear Regression	0.8524
Ridge Regression	0.8519
MultiTaskElasticNet	0.8510
MultiTaskLasso	0.8503

We also plotted a bar chart to visually compare the models' performances:



This chart compares the model performance based on their R^2 scores. R^2 shows how well the model predicted compared to the actual data. The closer the values are to 1 the better the prediction. In his chart we can clearly see that every model made good predictions, especially the Decision Tree model which took the lead.

6. Comparison of Models

Overall we can see that non-linear models performed better than their counterparts models. This means that the data has non-linear patterns. A linear model assumes a straight line comparison between input and output, while a non-linear can be complex e.g. curved. This means the relationship changes across the input space.

Multi-Output vs Multi-Task Setting:

Multi-Output: They wrap a separate estimator for each output variable. That means each target is predicted independently (without information sharing). Usually in practice, they perform better when, each target variable benefits from a dedicated model. In this case this regression method was better.

Multi-Task: They're designed to predict multiple related outputs simultaneously. They can perform better in practice, when target variables are strongly related to each other and benefit from a shared structure, which wasn't the case in the given data.

Final considerations:

This project focused mainly on predicting two key indicators of football players: market value and wage. We used a variety of regressors under two different modelling settings: Multi-Output Regression and the Multi-Task Learning.

We started the project by cleaning the given dataset, where we selected the relevant features, applied one-hot encoding for features like position and foot and frequency encoding for the feature “Team”, standardized it and identified relationships between attributes. Then we set up the different types of models and the hyperparameters, to see which model would produce a better R2 score. After the training of the models, we came in conclusion that non-linear models were better than the linear models.